

The header features a decorative graphic at the top consisting of several overlapping, semi-transparent blue and light blue geometric shapes, primarily triangles and quadrilaterals, creating a layered, mountain-like effect. A thin, dark grey diagonal line runs from the bottom left towards the top right, passing behind the blue shapes.

University of Layah

Submitted By: Muhammad Mudassar Nazar

Submitted To: Ma'am Saima Sattar

Department: Computer Science

Semester : 6th

Subject: Compiler Construction

Roll No: 49

Date: 11/05/2026

Building a Simple Compiler in C Language: A Step-by-Step Guide

Key Concepts and Abbreviations

Before we start, here are some important terms and abbreviations:

Compiler: A program that translates high-level language code into machine code or an intermediate representation.

Lexer (Lexical Analysis): Breaks the source code into tokens, the smallest units of meaning like keywords, operators, and identifiers.

Parser (Syntax Analysis): Analyzes tokens and checks if they follow the language's grammar rules.

Semantic Analyzer (Semantic Analysis): Ensures logical correctness, like type checking and variable declarations.

Intermediate Code Generation (ICG): Produces an intermediate representation (IR) from the parsed code, making optimization easier.

Code Optimization: Improves the intermediate code to make it more efficient.

Code Generation: Converts optimized intermediate code into machine code or assembly language.

Token: A string of characters that represent the smallest unit in the programming language (e.g., keywords, operators).

Grammar: A set of rules that define the structure of the programming language.

Phases of a Compiler

A simple compiler can be broken down into several phases:

Lexical Analysis: Tokenizes the input source code.

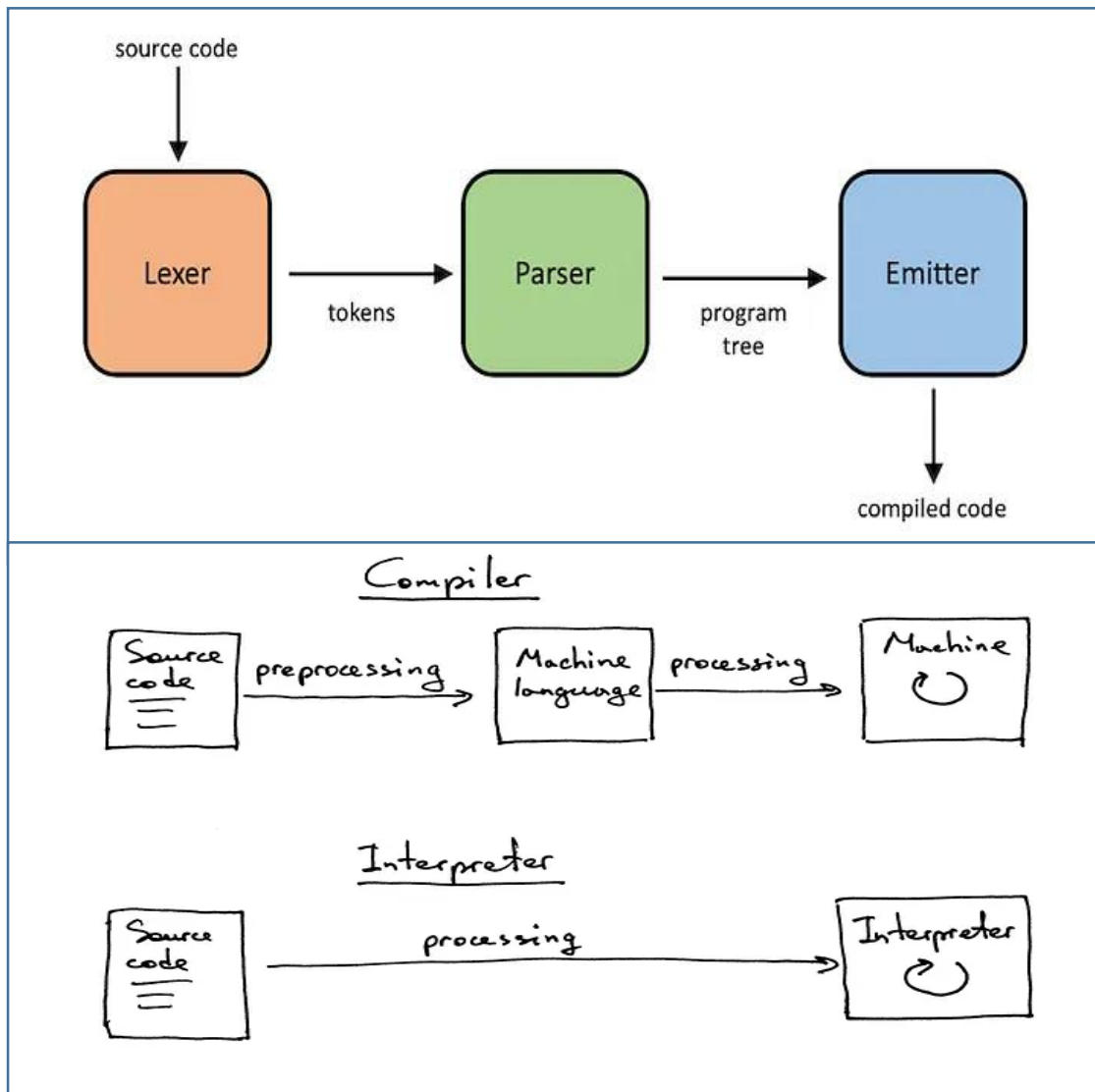
Syntax Analysis: Ensures the structure of the tokens is correct.

Semantic Analysis: Checks for logical correctness and validity.

Intermediate Code Generation (ICG): Converts the parse tree into an intermediate representation.

Code Optimization: Makes the intermediate code efficient.

Code Generation: Produces machine or assembly code from the intermediate code.



Step 1: Lexical Analysis

Lexical analysis is the first stage where the source code is broken down into meaningful tokens like keywords, identifiers, numbers, and operators. The lexer eliminates unnecessary whitespaces and comments.

Example: Lexical Analyzer in C

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
// Function to check if a string is a keyword
int isKeyword(char* str) {
    if (strcmp(str, "int") == 0 || strcmp(str, "return")
    == 0 || strcmp(str, "if") == 0) {
```

```
        return 1;
    }
    return 0;
}
// Lexer function to tokenize the input
void lexer(char* code) {
    char token[100];
```

```

int index = 0;
for (int i = 0; code[i] != '\0'; i++) {
    if (isspace(code[i])) continue;
    // Check for identifiers or keywords
    if (isalpha(code[i])) {
        while (isalpha(code[i])) {
            token[index++] = code[i++];
        }
        token[index] = '\0';
        if (isKeyword(token)) {
            printf("Keyword: %s\n", token);
        } else {
            printf("Identifier: %s\n", token);
        }
    }
    index = 0;
    i--;
}
// Check for numbers
else if (isdigit(code[i])) {
    while (isdigit(code[i])) {

```

```

        token[index++] = code[i++];
    }
    token[index] = '\0';
    printf("Number: %s\n", token);
    index = 0;
    i--;
}
// Check for operators
else if (code[i] == '+' || code[i] == '-') {
    printf("Operator: %c\n", code[i]);
}
}
}
int main() {
    char code[] = "int x = 10 + 20;";

    lexer(code);
    return 0;
}

```

Step 2: Syntax Analysis

In this step, the parser checks whether the sequence of tokens adheres to the grammar rules of the language. This step ensures that tokens form valid expressions, such as `identifier = expression`;

Example: Simple Parser in C

```

#include <stdio.h>
#include <ctype.h>
#include <string.h>
// Check if character is an operator
int isOperator(char c) {
    return (c == '+' || c == '-' || c == '*' || c == '/');
}
// Check if the expression is valid
int isValidExpression(char* expression) {
    int i = 0;
    // Check for identifier
    if (!isalpha(expression[i])) {
        printf("Syntax Error: Expected an identifier\n");
        return 0;
    }
    i++;

```

```

    // Check for assignment operator
    if (expression[i] != '=') {
        printf("Syntax Error: Expected '=' operator\n");
        return 0;
    }
    i++;
    // Check if the expression is valid (consisting of
    numbers or operators)
    while (expression[i] != '\0' && expression[i] !=
';') {
        if (!isdigit(expression[i]) &&
!isOperator(expression[i])) {
            printf("Syntax Error: Invalid expression\n");
            return 0;
        }
        i++;
    }

```

```

}
// Check for semicolon at the end
if (expression[i] != ';') {
    printf("Syntax Error: Expected ';' \n");
    return 0;
}
printf("Syntax is valid.\n");

```

```

return 1;
}
int main() {
    char statement[] = "x = 10 + 5;";
    isValidExpression(statement);
    return 0;
}

```

Step 3: Semantic Analysis

The next phase ensures that the tokens have correct meaning. For instance, variables must be declared before being used, and types must match in assignments.

Example: Semantic Analyzer in C

```

#include <stdio.h>
#include <string.h>
// Check if the variable has been declared
int isDeclared(char* variable, char variables[][10], int
size) {
    for (int i = 0; i < size; i++) {
        if (strcmp(variable, variables[i]) == 0) {
            return 1;
        }
    }
    return 0;
}
// Semantic analyzer to check for undeclared variables
void semanticAnalyzer(char* statement, char
variables[][10], int size) {
    char variable[10];
    sscanf(statement, "%s", variable);

```

```

    if (!isDeclared(variable, variables, size)) {
        printf("Semantic Error: Variable '%s' not
declared\n", variable);
    } else {
        printf("No semantic errors.\n");
    }
}
int main() {
    char variables[2][10] = { "x", "y" };
    char statement[] = "x = 10 + y;";
    semanticAnalyzer(statement, variables, 2);
    return 0;
}

```

Step 4: Intermediate Code Generation (ICG)

Once the code passes syntax and semantic analysis, the next step is to generate an intermediate representation (IR). This IR can be used to optimize and then generate final machine code. For simplicity, we'll generate a basic IR that translates high-level operations into a lower-level representation.

Example: Intermediate Code Generation in C

```

#include <stdio.h>
void generateIntermediateCode(char* statement) {

```

```

    char variable[10];
    int operand1, operand2;

```

```

char operator;
sscanf(statement, "%s = %d %c %d", variable,
&operand1, &operator, &operand2);
// Print simple three-address code
printf("t1 = %d\n", operand1);
printf("t2 = %d\n", operand2);
printf("t3 = t1 %c t2\n", operator);

```

```

printf("%s = t3\n", variable);
}
int main() {
char statement[] = "x = 10 + 20;";
generateIntermediateCode(statement);
return 0;
}

```

Step 5: Code Optimization (Optional)

Once intermediate code is generated, the compiler can apply various optimization techniques to improve performance. For example, it can eliminate common subexpressions, simplify expressions, and reduce unnecessary instructions.

Step 6: Code Generation

Finally, the optimized intermediate code is converted into target machine code or assembly. This part is highly specific to the target machine's architecture. In our case, we can generate pseudo-assembly instructions:

```

#include <stdio.h>
void generateMachineCode(char* statement) {
    char variable[10];
    int operand1, operand2;
    char operator;
    sscanf(statement, "%s = %d %c %d", variable, &operand1, &operator, &operand2);
    printf("MOV R1, #%d\n", operand1); // Load operand1 into R1
    printf("MOV R2, #%d\n", operand2); // Load operand2 into R2
    printf("%c R1, R2\n", operator); // Perform the operation
    printf("MOV %s, R1\n", variable); // Store the result in the variable
}
int main() {
    char statement[] = "x = 10 + 20;";
    generateMachineCode(statement);
    return 0;
}

```

Conclusion

Building a compiler, even a basic one, is a rewarding project that deepens your understanding of programming languages, computer architecture, and software development. The steps we covered here — lexical analysis, syntax analysis, semantic analysis, intermediate code generation, and machine code generation — form the backbone of real-world compilers.